# bene, in base ai concetti definiti in questo ragionamento, e tenuto conto del fatto che 40.services.sh deve essere un resolver + generatore non contenente logica specifica per servizio, mostrami una pseudo implementazione di questo script utilizzabile come punto di partenza per l'implementazione concreta dello stesso

Ecco una pseudo-implementazione in Bash, pensata come **resolver + generatore puro**: legge i manifest in `services/<type>/service.yaml`, risolve alias, parametri, immagini, volumi e porte, quindi emette un `compose.services.yaml` e un elenco di profili. Non contiene logica PostgreSQL/Redis: quella resta nei manifest e negli script per servizio. Compose combina più file `-f` nell'ordine fornito, quindi questo file può essere aggiunto dopo il Compose di base e l'eventuale override utente.[1]

### Contratto dello script

```
40.services.sh
  input:
    --target Python/foo
    --project-name dev.python.foo
    --with db,cache
    --services-file .docker-dev/generated/compose.services.yaml
    --profiles-file .docker-dev/generated/profiles
    --build-missing            # opzionale
    --dry-run                  # opzionale

  output:
    - compose.services.yaml
    - profiles (una per riga)
    - exit 0 oppure errore
```

### Pseudo-implementazione

```
#!/usr/bin/env bash
set -euo pipefail

# ------------------------------------------------------------------------------
# Configurazione
# ------------------------------------------------------------------------------
```

```
DOCKER_DEV_ROOT="${DOCKER_DEV_ROOT:-$(cd "$(dirname "${BASH_SOURCE[^0]}")/../.." && p
SERVICES_ROOT="${DOCKER_DEV_ROOT}/services"

TARGET=""
PROJECT_NAME=""
WITH=""
SERVICES_FILE=""
PROFILES_FILE=""
BUILD_MISSING=false
DRY_RUN=false

die() {
  printf 'docker-dev services: %s\n' "$*" >&2
  exit 1
}

usage() {
  cat <<'EOF'
Usage:
  40.services.sh \
    --target Python/foo \
    --project-name dev.python.foo \
    --with db,cache \
    --services-file PATH \
    --profiles-file PATH \
    [--build-missing] \
    [--dry-run]
EOF
}

# ------------------------------------------------------------------------------
# Parsing argomenti
# ------------------------------------------------------------------------------

while [[ $# -gt 0 ]]; do
  case "$1" in
    --target)
      TARGET="${2:-}"
      shift 2
      ;;
    --project-name)
      PROJECT_NAME="${2:-}"
      shift 2
      ;;
    --with)
      WITH="${2:-}"
      shift 2
      ;;
    --services-file)
      SERVICES_FILE="${2:-}"
      shift 2
      ;;
    --profiles-file)
      PROFILES_FILE="${2:-}"
      shift 2
```

```
      ;;
    --build-missing)
      BUILD_MISSING=true
      shift
      ;;
    --dry-run)
      DRY_RUN=true
      shift
      ;;
    -h|--help)
      usage
      exit 0
      ;;
    *)
      usage
      die "unknown argument: $1"
      ;;
  esac
done

[[ -n "$TARGET" ]] || die "--target is required"
[[ -n "$PROJECT_NAME" ]] || die "--project-name is required"
[[ -n "$SERVICES_FILE" ]] || die "--services-file is required"
[[ -n "$PROFILES_FILE" ]] || die "--profiles-file is required"

PROJECT_DIR="$PWD"
GENERATED_DIR="$(dirname "$SERVICES_FILE")"
mkdir -p "$GENERATED_DIR"

# ------------------------------------------------------------------------------
# Utility
# ------------------------------------------------------------------------------

# Estrae un valore scalare semplice da service.yaml.
# Per una implementazione reale: yq, python + PyYAML, jq dopo conversione,
# oppure un parser YAML minimale dedicato.
yaml_get() {
  local file="$1"
  local key="$2"
  local default="${3:-}"

  local value
  value="$(awk -v key="$key" '
    $0 ~ "^"key":" {
      sub("^"key":[[:space:]]*", "")
      gsub(/^["'\'' ]+|["'\'' ]+$/, "")
      print
      exit
    }
  ' "$file")"

  printf '%s' "${value:-$default}"
}

# Estrae il primo elemento di una lista YAML semplice:
# profiles:
```

```
#   - db
#   - full
yaml_get_list() {
  local file="$1"
  local key="$2"

  awk -v key="$key" '
    $0 ~ "^"key":" { inside=1; next }
    inside && /^[[:space:]]*-/ {
      line=$0
      sub(/^[[:space:]]*-[[:space:]]*/, "", line)
      gsub(/^["'\'' ]+|["'\'' ]+$/, "", line)
      print line
      next
    }
    inside && /^[^[:space:]-]/ { exit }
  ' "$file"
}

command_exists() {
  command -v "$1" >/dev/null 2>&1
}

# Verifica minima: in produzione userei yq o Python/PyYAML.
# Qui awk è solo un segnaposto per mantenere la pseudo-implementazione autonoma.
validate_yaml_tool() {
  if command_exists yq; then
    YAML_TOOL="yq"
  elif command_exists python3; then
    YAML_TOOL="python3"
  else
    die "yq or python3 is required to parse service manifests"
  fi
}

# ------------------------------------------------------------------------------
# Alias e risoluzione servizi
# ------------------------------------------------------------------------------

# Alias logici stabili per l'utente.
# Il valore è il tipo di servizio gestito in services/<type>/.
resolve_alias() {
  case "$1" in
    db)       echo "postgres" ;;
    cache)    echo "redis" ;;
    mail)     echo "mailpit" ;;
    object)   echo "minio" ;;
    *)        echo "$1" ;;
  esac
}

# Formato richiesto:
#   --with db,cache
#   --with postgres,redis
#   --with db:tag=16,host_port=15432
parse_requested_services() {
```

```
  local raw="$1"
  local item name params

  REQUESTED_NAMES=()
  REQUESTED_PARAMS=()

  [[ -z "$raw" ]] && return 0

  IFS=',' read -r -a items <<< "$raw"

  for item in "${items[@]}"; do
    [[ -z "$item" ]] && continue

    name="${item%%:*}"
    params="${item#*:}"

    [[ "$item" == "$name" ]] && params=""

    REQUESTED_NAMES+=("$name")
    REQUESTED_PARAMS+=("$params")
  done
}

# ------------------------------------------------------------------------
# Lettura manifest servizio
# ------------------------------------------------------------------------

load_service_manifest() {
  local service_type="$1"
  local manifest="${SERVICES_ROOT}/${service_type}/service.yaml"

  [[ -f "$manifest" ]] || die "unknown service type: $service_type (${manifest} not f

  SERVICE_MANIFEST="$manifest"
  SERVICE_TYPE="$service_type"

  SERVICE_NAME="$(yaml_get "$manifest" "name" "$service_type")"
  SERVICE_IMAGE_BASE="$(yaml_get "$manifest" "image" "")"
  SERVICE_DEFAULT_TAG="$(yaml_get "$manifest" "default_tag" "latest")"

  [[ -n "$SERVICE_IMAGE_BASE" ]] || \
    die "service ${service_type}: missing image in ${manifest}"
}

# ------------------------------------------------------------------------
# Parametri per istanza servizio
# ------------------------------------------------------------------------

# Valori di default; ogni servizio può fornire un proprio init/config script.
init_service_params() {
  SERVICE_TAG="$SERVICE_DEFAULT_TAG"
  SERVICE_HOST_PORT=""
  SERVICE_DATA_VOLUME=""
  SERVICE_EXTRA_ENV=""
}
```

```
parse_service_params() {
  local raw="$1"
  local key value

  [[ -z "$raw" || "$raw" == "$1" ]] && return 0

  IFS=',' read -r -a pairs <<< "$raw"

  for pair in "${pairs[@]}"; do
    [[ -z "$pair" ]] && continue

    key="${pair%%=*}"
    value="${pair#*=}"

    case "$key" in
      tag)
        SERVICE_TAG="$value"
        ;;
      host_port)
        SERVICE_HOST_PORT="$value"
        ;;
      volume)
        SERVICE_DATA_VOLUME="$value"
        ;;
      env)
        SERVICE_EXTRA_ENV="$value"
        ;;
      *)
        die "unknown parameter '${key}' for service ${SERVICE_TYPE}"
        ;;
    esac
  done
}

# ------------------------------------------------------------------------------
# Immagini e build
# ------------------------------------------------------------------------------

service_image() {
  printf '%s:%s' "$SERVICE_IMAGE_BASE" "$SERVICE_TAG"
}

image_exists() {
  local image="$1"
  docker image inspect "$image" >/dev/null 2>&1
}

build_service_image() {
  local image="$1"
  local service_dir="${SERVICES_ROOT}/${SERVICE_TYPE}"

  if [[ ! -f "${service_dir}/templates/Dockerfile" ]]; then
    die "service ${SERVICE_TYPE} cannot be built: missing Dockerfile"
  fi

  if [[ ! -x "${service_dir}/bin/20.build.sh" ]]; then
```

```
    die "service ${SERVICE_TYPE} cannot be built: missing executable 20.build.sh"
  fi

  printf 'Building service image %s\n' "$image" >&2

  "${service_dir}/bin/20.build.sh" \
    --service-type "$SERVICE_TYPE" \
    --image "$image" \
    --context "${service_dir}/templates"
}

ensure_service_image() {
  local image
  image="$(service_image)"

  if image_exists "$image"; then
    return 0
  fi

  if [[ "$BUILD_MISSING" == true ]]; then
    build_service_image "$image"
    return 0
  fi

  die "image ${image} not found; run: docker-dev --build-service=${SERVICE_TYPE}"
}

# ---------------------------------------------------------------------------
# Volumi
# ---------------------------------------------------------------------------

ensure_service_volume() {
  local volume_name="$1"
  local setup_script="${SERVICES_ROOT}/${SERVICE_TYPE}/bin/10.setup-volumes.sh"

  [[ -z "$volume_name" ]] && return 0

  if [[ -x "$setup_script" ]]; then
    "$setup_script" \
      --project-name "$PROJECT_NAME" \
      --service-type "$SERVICE_TYPE" \
      --volume-name "$volume_name"
  else
    docker volume inspect "$volume_name" >/dev/null 2>&1 || \
      docker volume create "$volume_name" >/dev/null
  fi
}

# ---------------------------------------------------------------------------
# Generazione Compose
# ---------------------------------------------------------------------------

write_header() {
  cat > "$SERVICES_FILE" <<EOF
# Generated by docker-dev 40.services.sh
# Source target: ${TARGET}
```

```
# Project: ${PROJECT_NAME}
# Do not edit manually.

services:
EOF

  : > "$PROFILES_FILE"
}

write_service() {
  local logical_name="$1"
  local image
  image="$(service_image)"

  cat >> "$SERVICES_FILE" <<EOF
  ${logical_name}:
    image: ${image}
    profiles:
      - ${SERVICE_TYPE}
    restart: unless-stopped
EOF

  # Healthcheck dichiarato nel manifest.
  # In una implementazione reale, il template YAML del servizio è preferibile:
  # qui si mostra solo il contratto.
  if [[ -n "$(yaml_get "$SERVICE_MANIFEST" "healthcheck" "")" ]]; then
    cat >> "$SERVICES_FILE" <<EOF
    healthcheck:
      test: ${SERVICE_HEALTHCHECK_TEST}
      interval: ${SERVICE_HEALTHCHECK_INTERVAL}
      timeout: ${SERVICE_HEALTHCHECK_TIMEOUT}
      retries: ${SERVICE_HEALTHCHECK_RETRIES}
EOF
  fi

  # Porte opzionali, solo se richieste esplicitamente o configurate.
  if [[ -n "$SERVICE_HOST_PORT" ]]; then
    cat >> "$SERVICES_FILE" <<EOF
    ports:
      - "127.0.0.1:${SERVICE_HOST_PORT}:${SERVICE_CONTAINER_PORT}"
EOF
  fi

  # Volumi persistenti.
  if [[ -n "$SERVICE_DATA_VOLUME" ]]; then
    cat >> "$SERVICES_FILE" <<EOF
    volumes:
      - ${SERVICE_DATA_VOLUME}:${SERVICE_DATA_MOUNT}
EOF
  fi

  # Variabili d'ambiente specifiche del servizio.
  if [[ -n "$SERVICE_ENVIRONMENT" ]]; then
    cat >> "$SERVICES_FILE" <<EOF
    environment:
${SERVICE_ENVIRONMENT}
```

```
EOF
  fi

  printf '%s\n' "$SERVICE_TYPE" >> "$PROFILES_FILE"
}

write_dev_dependencies() {
  local logical_name="$1"
  local connection_env="$2"
  local condition="${3:-service_started}"

  cat >> "$SERVICES_FILE" <<EOF

  dev:
    environment:
${connection_env}
    depends_on:
      ${logical_name}:
        condition: ${condition}
EOF
}

# ------------------------------------------------------------------------
# Emissione finale
# ------------------------------------------------------------------------

emit_service() {
  local index="$1"
  local requested_name="${REQUESTED_NAMES[$index]}"
  local requested_params="${REQUESTED_PARAMS[$index]}"

  local logical_name="$requested_name"
  local service_type
  service_type="$(resolve_alias "$requested_name")"

  load_service_manifest "$service_type"
  init_service_params
  parse_service_params "$requested_params"

  # Questi valori, in una implementazione reale, provengono dal manifest
  # attraverso un parser YAML affidabile oppure da un template per servizio.
  SERVICE_CONTAINER_PORT="$(yaml_get "$SERVICE_MANIFEST" "container_port" "")"
  SERVICE_DATA_MOUNT="$(yaml_get "$SERVICE_MANIFEST" "data_mount" "")"
  SERVICE_HEALTHCHECK_TEST="$(yaml_get "$SERVICE_MANIFEST" "healthcheck_test" "")"
  SERVICE_HEALTHCHECK_INTERVAL="$(yaml_get "$SERVICE_MANIFEST" "healthcheck_interval"
  SERVICE_HEALTHCHECK_TIMEOUT="$(yaml_get "$SERVICE_MANIFEST" "healthcheck_timeout" "
  SERVICE_HEALTHCHECK_RETRIES="$(yaml_get "$SERVICE_MANIFEST" "healthcheck_retries" "

  # Il manifest può definire un volume di default.
  if [[ -z "$SERVICE_DATA_VOLUME" ]]; then
    SERVICE_DATA_VOLUME="$(yaml_get "$SERVICE_MANIFEST" "default_volume" "")"
    [[ -n "$SERVICE_DATA_VOLUME" ]] && \
      SERVICE_DATA_VOLUME="${SERVICE_DATA_VOLUME//\{project\}/${PROJECT_NAME}}"
  fi

  ensure_service_image
```

```
  ensure_service_volume "$SERVICE_DATA_VOLUME"

  # La generazione reale dovrebbe usare un template Compose per servizio:
  # services/<type>/templates/compose.yaml.j2
  #
  # Questo blocco è un fallback semplificato per illustrare il flusso.
  write_service "$logical_name"

  # Le variabili di connessione dovrebbero essere fornite dal servizio stesso,
  # ad esempio services/postgres/templates/connection.env.j2.
  local connection_env
  connection_env="$(generate_connection_env "$logical_name")"

  local condition="service_started"
  [[ -n "$SERVICE_HEALTHCHECK_TEST" ]] && condition="service_healthy"

  write_dev_dependencies "$logical_name" "$connection_env" "$condition"
}

generate_connection_env() {
  local logical_name="$1"

  # Segnaposto: il valore reale deve provenire dal manifest/template del servizio.
  case "$SERVICE_TYPE" in
    postgres)
      cat <<EOF
      DATABASE_URL: postgres://app:app@${logical_name}:5432/app
EOF
      ;;
    redis)
      cat <<EOF
      REDIS_URL: redis://${logical_name}:6379/0
EOF
      ;;
    *)
      cat <<EOF
      ${SERVICE_TYPE^^}_URL: ${logical_name}
EOF
      ;;
  esac
}

# ------------------------------------------------------------------------------
# Main
# ------------------------------------------------------------------------------

validate_yaml_tool
parse_requested_services "$WITH"

if [[ ${#REQUESTED_NAMES[@]} -eq 0 ]]; then
  printf '# No optional services requested.\n' > "$SERVICES_FILE"
  : > "$PROFILES_FILE"
  exit 0
fi

write_header
```

```
for i in "${!REQUESTED_NAMES[@]}"; do
  emit_service "$i"
done

if [[ "$DRY_RUN" == true ]]; then
  printf 'Generated: %s\n' "$SERVICES_FILE" >&2
  printf 'Profiles: %s\n' "$(tr '\n' ' ' < "$PROFILES_FILE")" >&2
  exit 0
fi

exit 0
```

### Esempio di output

Con:

```
40.services.sh \
  --target Python/foo \
  --project-name dev.python.foo \
  --with db,cache \
  --services-file .docker-dev/generated/compose.services.yaml \
  --profiles-file .docker-dev/generated/profiles
```

l'output atteso sarebbe:

```
services:
  db:
    image: dev.postgres:16
    profiles:
      - postgres
    restart: unless-stopped
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d app"]
      interval: 5s
      timeout: 3s
      retries: 10
    ports:
      - "127.0.0.1:15432:5432"
    volumes:
      - data.dev.postgres.foo:/var/lib/postgresql/data
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app

  cache:
    image: dev.redis:7
    profiles:
      - redis
    restart: unless-stopped
    ports:
      - "127.0.0.1:16379:6379"
```

```
  dev:
    environment:
      DATABASE_URL: postgres://app:app@db:5432/app
      REDIS_URL: redis://cache:6379/0
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started

volumes:
  data.dev.postgres.foo:
    external: true
```

E `profiles`:

```
postgres
redis
```

### Miglioramenti da fare nella versione reale

- **Sostituisci `awk` con un parser YAML affidabile**: `yq`, oppure Python + PyYAML. Lo script Bash sopra usa `awk` solo come segnaposto; per manifest reali è troppo fragile.
- **Usa template Compose per servizio**: `services/postgres/templates/compose.yaml.j2` dovrebbe contenere l'intero blocco del servizio. `40.services.sh` si limita a renderizzare e unire i frammenti.
- **Non generare `dev` nel file servizi se non necessario**: preferisci che il template del linguaggio dichiari hook tipo `DDEV_SERVICE_ENV` e `DDEV_SERVICE_DEPENDS_ON`, oppure genera un frammento separato per `dev`.
- **Valida il risultato**: esegui sempre `docker compose -f ... config --quiet` prima di `up`.
- **Determinismo**: salva un hash di manifest, parametri, versione template e immagine in `.docker-dev/generated/services.lock`. Se cambia, rigenera e avvisa.
- **Evita conflitti di porta**: assegna porte host dinamiche oppure verifica la disponibilità prima della generazione.
- **Non fidarti dei parametri CLI per percorsi o comandi**: valida rigorosamente `tag`, `host_port`, `volume` e `env`.

Il punto architetturale è che `40.services.sh` deve sapere solo come: leggere un manifest, risolvere alias e parametri, invocare build/setup per servizio, renderizzare un frammento Compose e dichiarare i profili. La conoscenza concreta di PostgreSQL, Redis o Mailpit vive interamente in `services/<type>/`. I profili Compose permettono poi di attivare selettivamente questi servizi senza avviare tutto lo stack. [2]

⁂

1. https://docs.docker.com/compose/how-tos/multiple-compose-files/include.md

2. https://forums.docker.com/t/direct-command-to-compose-up-down-a-profile-without-up-down-default/152172
3. https://docs.docker.com/compose/how-tos/multiple-compose-files/extends/
4. https://docs.docker.com/compose/how-tos/multiple-compose-files/merge/
5. https://event-driven.io/en/docker_compose_profiles/
6. https://jacar.es/en/docker-compose-profiles/
7. https://tourmalinecore.com/articles/docker-compose-profiles-templates
8. https://www.linkedin.com/learning/learning-docker-compose-simplified-multi-container-application-management/docker-compose-profiles
9. https://github.com/docker-archive/compose-cli/issues/1964
10. https://docs.docker.com/compose/how-tos/profiles/
11. https://docs.docker.com/reference/compose-file/merge/
12. https://docs.docker.com/reference/compose-file/profiles/
13. https://docs.docker.com/reference/cli/docker/compose/
14. https://docs.docker.com/compose/
15. https://docs.docker.com/reference/compose-file/